

Software Requirements Specification for Online Vehicle Rental System

Version 1.0 approved

**Prepared by Anuj Lulu
Department of Computer Science
July 21, 2026**

TABLE OF CONTENTS

1. INTRODUCTION.....	4
1.1 Purpose.....	4
1.2 Scope.....	4
1.3 Definitions and Acronyms.....	4
1.4 References.....	5
1.5 Document Overview.....	5
2. OVERALL DESCRIPTION.....	5
2.1 Product Perspective.....	5
2.2 Product Functions.....	5
2.3 User Classes and Characteristics.....	5
2.4 Operating Environment.....	6
2.5 Design and Implementation Constraints.....	6
2.6 Assumptions and Dependencies.....	6
3. EXTERNAL INTERFACE REQUIREMENTS.....	7
3.1 User Interfaces.....	7
3.2 Hardware Interfaces.....	7
3.3 Software Interfaces.....	7
3.4 Communication Interfaces.....	7
4. FUNCTIONAL REQUIREMENTS.....	8
4.1 User Management and Verification.....	8
4.2 Vehicle Listing and Inventory Lifecycle.....	8
4.3 Booking Engine and Concurrency Controls.....	8
4.4 Payment Orchestration and Fault Recovery.....	9
4.5 Cancellation, Refueling, and Asset Guardrails.....	9
4.6 Telemetry, Safety SOS, and Integrity Guardrails.....	10
4.7 Automated System Notifications.....	10
5. NON-FUNCTIONAL REQUIREMENTS.....	11
5.1 Performance and Scalability.....	11
5.2 Availability and Reliability.....	11
5.3 Security and Privacy.....	11
5.4 Usability and Accessibility.....	12
5.5 Auditability and Data Integrity.....	12
6. BUSINESS RULES.....	12
7. DETAILED USE CASES.....	13
7.1 Use Case 1: Customer Books Vehicle (UC-1.0).....	13
7.2 Use Case 2: Owner Lists Vehicle (UC-2.0).....	14
7.3 Use Case 3: Administrator Approves Listing (UC-3.0).....	15
7.4 Use Case 4: Customer Reports Accident (UC-4.0).....	16

8. UML SYSTEM ARTIFACTS.....	17
8.1 Use Case Diagram (Textual Representation).....	17
8.2 Activity Diagram: Customer Booking to Return Workflow.....	17
8.3 Sequence Diagram: Real-Time Booking Processing Execution.....	17
8.4 State Diagram: Vehicle Asset Lifecycle Transitions.....	18
9. DATABASE REQUIREMENTS.....	18
9.1 Schema Design Relationship Descriptions.....	18
9.2 Complete Entity Attribute Mappings.....	19
10. REQUIREMENTS TRACEABILITY MATRIX (RTM).....	20
11. ACCEPTANCE CRITERIA.....	20
AC-1.0: User Verification Performance.....	20
AC-2.0: Double-Booking Prevention Under Load.....	21
AC-3.0: SOS Disaster Notification Latency.....	21
AC-4.0: Immutable Review Guardrails.....	21
12. FUTURE ENHANCEMENTS.....	21
12.1 AI Demand Prediction and Dynamic Pricing.....	21
12.2 Predictive Maintenance and IoT Diagnostics.....	21
12.3 EV Charging Network Integration.....	21

1. INTRODUCTION

1.1 Purpose

This document specifies the complete, definitive functional, non-functional, and interface requirements for the Online Vehicle Rental System (OVRs) Version 1.0.0. This specification is designed to guide the engineering team during implementation, the quality assurance team during validation testing, and system administrators during operational deployment.

1.2 Scope

The OVRs is a web and mobile-based software ecosystem that facilitates automated, on-demand vehicle rentals. In-scope core capabilities include cross-platform user onboarding with automated third-party Government ID and Driving License verification; multi-class vehicle inventory tracking (Cars, Two-wheelers, Bicycles) with localized parking hub allocation; highly concurrent booking engine featuring optimistic and pessimistic locking protocols to eliminate double-bookings; automated payment orchestration supporting multi-modal gateways (UPI, Cards, Net Banking, Wallets) with transactional failure recovery; live vehicle tracking using embedded IoT GPS transponders, featuring an immediate SOS emergency dispatch system; and decentralized vehicle owner modules with an automatic 80/20 revenue-sharing split engine and immutable rating histories. Out-of-scope capabilities include physical vehicle manufacturing, hardware design of IoT GPS transponders, and legal underwriting of insurance policies.

1.3 Definitions and Acronyms

- **ACID:** Atomicity, Consistency, Isolation, Durability. A set of properties that guarantee database transactions are processed reliably.
- **API:** Application Programming Interface.
- **BR:** Business Rule.
- **FR:** Functional Requirement.
- **GPS:** Global Positioning System.
- **JWT:** JSON Web Token. An open standard for securely transmitting information between parties as a JSON object.
- **NFR:** Non-Functional Requirement.
- **OVRs:** Online Vehicle Rental System.
- **PCI-DSS:** Payment Card Industry Data Security Standard.
- **SOS:** international distress signal / Save Our Souls (Emergency notification trigger).

- **SRS:** Software Requirements Specification.
- **UC:** Use Case.

1.4 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- ISO/IEC/IEEE 29148:2018, Systems and software engineering — Life cycle processes — Requirements engineering.
- Payment Card Industry (PCI) Data Security Standard, Version 4.0.
- Information Technology Act, 2000 (India) for digital signature and data privacy baselines.

1.5 Document Overview

This document follows standard software engineering conventions. Section 2 describes the structural perspective, user classes, operating constraints, and foundational assumptions. Section 3 delineates external interface interactions. Section 4 provides an exhaustive breakdown of itemized Functional Requirements, followed by Section 5 detailing Non-Functional performance and security baselines. Section 6 presents formal Business Rules, Section 7 models core Use Cases, Section 8 establishes UML system architectures, Section 9 contains data design schemas, and Sections 10 and 11 detail the Traceability Matrix and Acceptance Criteria.

2. OVERALL DESCRIPTION

2.1 Product Perspective

The OVRS is an independent, newly engineered software product. It interacts with several external services, including payment gateways, identity validation systems, mapping networks, and onboard vehicle IoT tracking systems.

2.2 Product Functions

The high-level functions of the system are categorized by operational domains:

- **Identity and Asset Governance:** Validates users, records owner down payments, and manages vehicle lifecycles.
- **State-Controlled Inventory Management:** Exposes available assets grouped by hub location, asset configuration, and fuel statuses.
- **Transactional Reservation Engine:** Computes pricing schedules, processes current and scheduled reservations, and manages isolation levels to prevent double-bookings.

- **Telemetry and Security Orchestration:** Processes incoming streaming spatial data coordinates from vehicles and handles SOS emergency signals.
- **Financial Clearinghouse:** Computes platform splits, manages security deposits, processes late fees, and handles transaction retries.

2.3 User Classes and Characteristics

- **Customer:** End-users who search for, book, track, and operate rented vehicles. They have varying technical skills and interact primarily through mobile applications or responsive web interfaces.
- **Vehicle Owner:** Third-party partners who list their vehicles on the platform, deposit security bonds, track their assets' rental histories, and view financial payouts.
- **Parking Worker:** On-ground hub operators who verify vehicle identification numbers (VIN), inspect physical vehicles post-rental, and update readiness states via a mobile application.
- **Administrator:** Centralized operators with full access to systemic data. They monitor real-time metrics, review flagged ratings, handle emergency incidents, and audit platform finances.

2.4 Operating Environment

- **Client Applications:** Android OS (v9.0 and above), iOS (v14.0 and above), and modern web browsers (Chrome v100+, Safari v15+, Firefox v100+).
- **Application Server:** Containerized microservices deployed within cloud networks (AWS/GCP), running Linux Ubuntu 22.04 LTS environments.
- **Database Cluster:** Multi-AZ relational instance (PostgreSQL v15+) with a geospatial extension (PostGIS) and an in-memory cache system (Redis v7.0+).

2.5 Design and Implementation Constraints

- **C-1:** All database operations regarding financial state transitions or vehicle scheduling states must strictly enforce serializable transaction isolation levels or row-level explicit blocks to eliminate data race conditions.
- **C-2:** The system cannot persist raw credit card credentials, pins, or CVVs. It must use tokenized payment links compliant with PCI-DSS Level 1 standards.
- **C-3:** All communication channels must encrypt data in transit using TLS 1.3, and data at rest must use AES-256 bit encryption algorithms.

2.6 Assumptions and Dependencies

- **A-2.1:** It is assumed that vehicles registered by owners are equipped with an operational, battery-backed IoT GPS transponder that broadcasts cellular data coordinates over HTTP/MQTT protocols every 5000 milliseconds.
- **A-2.2:** The system depends on the uninterrupted availability of third-party government verification APIs and payment clearing networks.

3. EXTERNAL INTERFACE REQUIREMENTS

3.1 User Interfaces

All customer interfaces must adhere to a mobile-first, responsive design framework with interactive elements designed for easy touch operation (minimum hit target of 48x48 pixels). The administrative portal must utilize a desktop-optimized layout featuring a multi-pane dashboard, real-time data tables, canvas-rendered map layers, and text search filtering.

3.2 Hardware Interfaces

The application must interface with client-side mobile hardware components: Biometric readers (FaceID/TouchID), GPS modules for locating the nearest hub, and high-resolution camera lenses for scanning identification barcodes and document photos. The system must interface with vehicle embedded telemetry boards via cellular networks to capture vehicle state information (GPS coordinates, engine ignition statuses, lock states, odometer readings, and internal battery charges).

3.3 Software Interfaces

- **Identity Verification Interface:** JSON over HTTPS RESTful API connecting to state vehicle registry databases and national identification systems.
- **Payment Gateway Interface:** Webhook and redirection API endpoints supporting secure handshakes with payment aggregators.
- **Mapping and Routing Engine:** Vector tile loading and geographic routing matrices for calculating walking distances to hubs and active vehicle route plotting.

3.4 Communication Interfaces

- **MQTT/WebSockets Protocol:** Employed for real-time, low-overhead communication between active vehicle GPS units, client maps, and the core tracking servers.
- **HTTPS Protocol:** Enforced for all application-level CRUD transactional operations.

- **SMTP/SMS Gateway Protocol:** Integrated via external communication brokers to dispatch immediate transactional notices and OTP (One-Time Password) confirmations.

4. FUNCTIONAL REQUIREMENTS

4.1 User Management and Verification

- **FR-1.1:** The system must provide distinct registration workflows for Customers, Vehicle Owners, and Parking Workers, collecting full legal names, email addresses, phone numbers, and physical addresses.
- **FR-1.2:** The system must process uploaded government identification documents via an automated verification API. It must verify the user's name, document authenticity, and date of birth.
- **FR-1.3:** The system must block any user from reserving a car or two-wheeler if their verified age is under 18 years, or if they fail to provide a valid, unexpired driving license verified by the state database.
- **FR-1.4:** The system must allow users who have reached a minimum verified age of 16 years to rent human-powered or electric bicycles without requiring a driving license.

4.2 Vehicle Listing and Inventory Lifecycle

- **FR-2.1:** Every vehicle item entry rendered to a user must contain the following fields: Photos, License Plate Number, Hourly Price, Fuel/Battery Type, Tank/Battery Capacity, Mileage, Comfort Rating, Age, Current Availability Status, Hub Geolocation, Owner Identifier, and Historical Reviews.
- **FR-2.2:** The system must provide real-time search filtering across the following criteria: Price Range, Vehicle Category (Car, Two-wheeler, Bicycle), Fuel/Power Source (Petrol, Diesel, CNG, Electric), Aggregate User Rating, Vehicle Age, Model Mileage, Hub Location Radius, and explicit Time-frame Availability windows.
- **FR-2.3:** The system must require a new Vehicle Owner to submit proof of vehicle registration, valid commercial insurance, and a roadworthiness certificate. The listing remains hidden until the owner pays a refundable security bond of ₹5,000.
- **FR-2.4:** Upon owner document clearance and deposit processing, the system must generate a physical staging task. A designated Parking Worker must verify the vehicle's serial number, perform an inspection check, and manually transition the asset's state within the system to stage it at a designated parking hub.

4.3 Booking Engine and Concurrency Controls

- **FR-3.1:** The system must support immediate on-demand rentals as well as reservations scheduled up to 30 days in advance.
- **FR-3.2:** If a requested booking duration is calculated to be under 12 hours, the system must enforce a mandatory reservation model. This model locks the vehicle selection to a specific license plate number, rather than a generic vehicle class.
- **FR-3.3:** Upon a user clicking the booking confirmation button, the system must immediately switch that vehicle's status to RESERVED_LOCKED. This prevents the asset from appearing in any concurrent user search results.
- **FR-3.4:** When thousands of simultaneous users request the final available vehicle asset at a given hub, the database transaction manager must use explicit row-level locking controls under a Serializable transaction isolation level. Only the initial transaction to clear the lock succeeds. All concurrent overlapping threads must fail immediately and receive a "Vehicle No Longer Available" error notification within 850 milliseconds.
- **FR-3.5:** After a vehicle is returned by a customer, the system must lock its state to IN_INSPECTION for a mandatory 30-minute window. The vehicle cannot be reserved by another customer until a Parking Worker completes a digital inspection checklist confirming cleaning, refueling, or recharging.

4.4 Payment Orchestration and Fault Recovery

- **FR-4.1:** The system must process payments through integrated, tokenized endpoints supporting UPI, Credit Cards, Debit Cards, Net Banking, and Mobile Wallets.
- **FR-4.2:** If a communication drop or client application crash occurs before the payment gateway issues a valid transaction token, the system must run an automatic cleanup job. This job invalidates the temporary reservation lock within 120 seconds and returns the vehicle asset back to active inventory.
- **FR-4.3:** If a payment attempt fails mid-checkout due to insufficient funds or banking network response timeouts, the core system must present the user with a remediation interface offering three options: Retry using a different card or token, switch instantly to alternate channels (e.g., UPI or digital wallet), or explicitly cancel the transaction.
- **FR-4.4:** The system must continuously compute temporal overages based on vehicle telemetry data. If a customer fails to return an asset by the scheduled completion time, the engine must transition the billing state to late mode, charging the standard hourly rate for every fractional hour until the vehicle passes verification at an authorized hub.

- **FR-4.5:** Upon successful completion of a rental transaction, the accounting module must calculate the revenue split. It must allocate exactly 80% of the net rental fee to the verified Vehicle Owner's escrow account, and route the remaining 20% to the platform's operational treasury ledger.

4.5 Cancellation, Refueling, and Asset Guardrails

- **FR-5.1:** The system must allow a customer to cancel a booking within a strict 15-minute window following initial payment confirmation. This triggers a full refund without penalization fees.
- **FR-5.2:** When a valid cancellation is processed, the system must invoke the payment gateway's refund API within 60 seconds. It must update the user's dashboard with an immutable reference ID and display an advisory note stating that funds will appear within 5 to 7 standard banking business days.
- **FR-5.3:** The system must ensure that a vehicle is handed over to a customer with a 100% full fuel tank or a fully charged battery pack. This status must be digitally verified and logged by a Parking Worker prior to handover.
- **FR-5.4:** Upon vehicle return, the system must read the fuel sensor or battery level telemetry data. If the asset is returned with less than a full tank or charge, the system must compute the missing fraction and automatically bill the customer's payment profile for the exact volume consumed, mapping it to current fuel or energy market rates.
- **FR-5.5:** The core system must assign financial and operational liability for routine maintenance, periodic servicing, and mechanical repairs directly to the vehicle owner. The system must automatically lock a vehicle out of inventory and transition it to `MAINTENANCE_REQUIRED` when its mileage data triggers set service intervals.

4.6 Telemetry, Safety SOS, and Integrity Guardrails

- **FR-6.1:** The system must process continuous stream inputs from vehicle GPS units. Administrators must have a global view of all active assets, while owners must be limited to viewing tracking streams for their specific registered vehicles.
- **FR-6.2:** The mobile application and vehicle firmware must restrict customers from pausing, masking, or altering geographic tracking coordinates during an active rental session.
- **FR-6.3:** The mobile application interface must feature an easily accessible, persistent one-touch SOS Emergency Button. When triggered, the system must prioritize this request over all other network traffic and broadcast an emergency packet containing precise coordinates, telemetry timestamp, vehicle VIN, user ID, and emergency contacts. It must also open an incident window on the

administrator console and generate an immediate push notification to emergency response networks.

- **FR-6.4:** When an accident incident is logged, the system must lock all vehicle data from the preceding 30 minutes to preserve evidence. It must also generate a guided interface for uploading damage photos, police report numbers, and third-party details, routing this data directly to the integrated insurance carrier's intake API.
- **FR-6.5:** The database must treat user ratings, textual reviews, and helpfulness upvotes/downvotes as append-only records. The system must prevent owners or users from deleting or modifying ratings. Any moderation request requires a multi-sign admin audit workflow, and a complete history of changes must be maintained for security reviews.

4.7 Automated System Notifications

- **FR-7.1:** The notification system must send updates across SMS, Email, and Push Notification channels based on changes in system state.
- **FR-7.2:** The notification engine must automatically generate messages for specific lifecycle events:
 - **FR-7.2.1 (Payment Success):** Dispatched to the Customer containing the Reservation ID, Hub Address, and a Dynamic QR Code for Pickup within 5 seconds.
 - **FR-7.2.2 (2-Hour Pickup Window):** Dispatched to the Customer containing Hub Directions and Staged Vehicle Identification Parameters within 15 minutes of the target window.
 - **FR-7.2.3 (1-Hour Return Window):** Dispatched to the Customer containing a Map Link to the Target Hub and Late-Fee Overtime Penalty Rates within 5 minutes of the target window.
 - **FR-7.2.4 (Rental Duration Alerts):** Dispatched to the Customer containing an Active Countdown Timer and Options to Request Extension within 60 seconds.
 - **FR-7.2.5 (Order Cancellation):** Dispatched to the Customer and Owner containing the Cancellation Log, Refund Amount, and Asset Release Notice within 10 seconds.

5. NON-FUNCTIONAL REQUIREMENTS

5.1 Performance and Scalability

- **NFR-1.1:** The system must process a minimum of 5,000 read transactions per second and 1,500 write/booking updates per second without exceeding a maximum database lock latency of 250 milliseconds.
- **NFR-1.2:** All standard catalog exploration queries, filter adjustments, and map rendering requests must deliver their payload within a maximum response time of 300 milliseconds under peak load conditions.

5.2 Availability and Reliability

- **NFR-2.1:** The core system architecture must maintain an availability metric of 99.99% over any given calendar month, excluding scheduled off-peak maintenance windows which cannot exceed 120 minutes per year.
- **NFR-2.2:** If external downstream components go offline, the application must implement a graceful degradation strategy. It must use circuit-breaker patterns to route requests through backup systems while keeping core search and inventory features operational.

5.3 Security and Privacy

- **NFR-3.1:** All data transmission must be secured using TLS 1.3 encryption. Sensitive user data, including driver's license details and identification hashes, must be encrypted at rest using AES-256 bit encryption keys managed by a cloud security module.
- **NFR-3.2:** The system must enforce strict role-based access controls. Administrative APIs must require multi-factor authentication (MFA) and must reject requests that lack valid cryptographically signed JWT tokens containing explicit permission flags.

5.4 Usability and Accessibility

- **NFR-4.1:** The user-facing web applications must comply with WCAG 2.1 Level AA accessibility standards, ensuring proper contrast ratios, screen reader compatibility, and clear text sizing for all elements.

5.5 Auditability and Data Integrity

- **NFR-5.1:** All ledger updates, payment changes, and balance transfers must be recorded into a write-once, append-only database log table. This log must prevent deletions or modifications by any system user, including top-level administrators.
- **NFR-5.2:** The system must store all historical geographic coordinates, speed metrics, and device alerts from active vehicles for a minimum of 365 days to assist with legal, insurance, and performance evaluations.

6. BUSINESS RULES

- **BR-6.1:** Customers may cancel an active booking for a full refund up to 15 minutes after making a payment. If a cancellation is requested after this 15-minute grace period has passed, the system must retain 100% of the booking deposit as a cancellation fee.
- **BR-6.2:** An active booking reserves a specific vehicle identification number (VIN) for rentals shorter than 12 hours. The system must prevent field staff or administrators from changing this specific vehicle assignment unless a verified mechanical breakdown is reported before pickup.
- **BR-6.3:** Every vehicle returned to a hub must enter a locked state for at least 30 minutes. The system must prevent this vehicle from being rented or appearing in search results until a parking worker completes the required digital inspection and cleaning checklist.
- **BR-6.4:** Vehicles must be handed over to customers with a completely full fuel tank or battery charge. If a vehicle is returned with less fuel or battery charge than at handover, the system must automatically bill the customer for the missing energy volume at current market rates, plus an operational refuel service charge of 15%.
- **BR-6.5:** All rental revenue must be calculated and split automatically at the end of each rental session. The platform must route exactly 80% of the net transaction value to the vehicle owner's account and retain 20% for platform operations.
- **BR-6.6:** The system must enforce age and documentation checks before allowing a booking. Users must be at least 18 years old and hold a verified, unexpired driver's license to reserve motorized vehicles. Users must be at least 16 years old to reserve bicycles.
- **BR-6.7:** To activate listings on the platform, owners must pay a refundable security bond of ₹5,000 per vehicle. The system must automatically hide an owner's vehicle listings if this security bond balance drops below ₹5,000 due to outstanding maintenance or safety penalties.

7. DETAILED USE CASES

7.1 Use Case 1: Customer Books Vehicle (UC-1.0)

- **Actors:** Customer, Payment Gateway, Notification System.
- **Preconditions:** The customer has completed account setup, passed identity and driving license verifications, and has an active account status.

- **Trigger:** The customer selects a specific vehicle from the search results and clicks the "Confirm Reservation" button.
- **Main Flow:**
 1. The customer selects an available vehicle, chooses a pickup time, and submits the reservation request.
 2. The system creates a temporary reservation lock on the vehicle asset, changing its status to RESERVED_LOCKED.
 3. The system calculates the total rental fee, factoring in duration, vehicle type, and tax rates.
 4. The system securely routes the checkout session to the external payment gateway.
 5. The customer enters valid payment credentials and authorizes the transaction.
 6. The payment gateway processes the payment and returns a success token to the system.
 7. The system updates the vehicle status to BOOKED and records an entry in the ledger.
 8. The system generates a reservation confirmation QR code and sends it via push notification and email.
- **Alternate Flow:** Step 5 (Payment Failure): The payment gateway rejects the transaction. The system notifies the user of the payment error and displays options to retry with an alternate payment method or cancel the request.
- **Exception Flow:** Step 2 (Database Overlap): Another user completes a checkout for the same vehicle at the exact same instant. The system catches the data collision, cancels the temporary lock, displays a "Vehicle unavailable" alert, and returns the customer to the search results page.
- **Postconditions:** The target vehicle is moved to a booked status, a payment record is appended to the ledger, and a booking QR code is generated for the customer.

7.2 Use Case 2: Owner Lists Vehicle (UC-2.0)

- **Actors:** Vehicle Owner, Administrative System, Parking Worker.
- **Preconditions:** The owner has completed registration and provided valid bank account information for automated payouts.
- **Trigger:** The owner clicks the "List New Asset" button on the owner management portal.

- **Main Flow:**

1. The owner enters the vehicle details, including the license plate, model, manufacturing year, and fuel specifications.
2. The owner uploads digital copies of the vehicle registration certificate, commercial insurance policy, and pollution clearance records.
3. The system checks the documents against basic registration rules and routes them to the administration queue for verification.
4. The system prompts the owner to submit the mandatory ₹5,000 refundable security bond.
5. The owner completes the payment transaction, and the system logs the security deposit.
6. An administrator reviews and approves the vehicle documents.
7. The system assigns a pickup task to a parking worker at the selected hub.
8. The parking worker verifies the vehicle's details on-site, installs the IoT tracking hardware, and flags the vehicle as ready for service.

- **Alternate Flow:** Step 6 (Document Rejection): The administrator identifies an expired insurance certificate. The system marks the listing as rejected, notifies the owner with details on the issue, and opens a re-upload interface.
- **Exception Flow:** Step 5 (Deposit Failure): The security bond payment fails or is abandoned. The system pauses the onboarding process, leaves the vehicle status as PENDING_DEPOSIT, and prevents the listing from routing to the administrator queue.
- **Postconditions:** The vehicle passes verification, is assigned to a physical hub, and its state is set to AVAILABLE within the platform repository.

7.3 Use Case 3: Administrator Approves Listing (UC-3.0)

- **Actors:** Administrator, Document Verification Engine.
- **Preconditions:** A vehicle owner has uploaded an asset profile along with all required legal and regulatory documentation.
- **Trigger:** A new document verification task appears in the administrative dashboard queue.
- **Main Flow:**
 1. The administrator opens the pending verification task to view the asset profile and documents.

2. The system runs an automated check against government registration APIs to confirm the license plate and vehicle identification number (VIN) match.
 3. The administrator inspects the digital images of the commercial insurance policy and pollution clearance records.
 4. The administrator verifies that the automated government API checks passed successfully.
 5. The administrator clicks the "Approve Asset Listing" button.
 6. The system updates the asset status to APPROVED_BY_ADMIN_STATE and logs the administrator ID.
 7. The system sends an automated notification to the owner confirming listing approval.
- **Alternate Flow:** Step 4 (Automated Check Failure): The government API returns a data mismatch for the VIN. The system automatically highlights the conflicting fields, alerts the administrator, and prompts them to route the task to a manual dispute review state.
 - **Postconditions:** The asset status transitions to an approved tier, making it eligible for physical staging and activation at a designated parking hub.

7.4 Use Case 4: Customer Reports Accident (UC-4.0)

- **Actors:** Customer, Administrator, Emergency Dispatch Network, Insurance Carrier API.
- **Preconditions:** The customer has an active, ongoing vehicle rental session.
- **Trigger:** The customer encounters an incident and taps the "Report Accident / SOS" button within the mobile application.
- **Main Flow:**
 1. The customer activates the emergency interface and taps the "Confirm Incident Reporting" prompt.
 2. The system immediately captures the vehicle's current GPS coordinates, speed metrics, and user profile data.
 3. The system sends an urgent incident alert to the administrator console and triggers local emergency dispatch services.
 4. The system locks all vehicle data and telemetry from the preceding 30 minutes into a secure data storage block.

5. The mobile application displays emergency instructions and opens a secure upload tool for collecting documentation.
 6. The customer uploads on-scene photos of vehicle damage and enters a summary text description.
 7. The system aggregates the telemetry logs, user statements, and images into an incident packet, then submits it directly to the insurance carrier's intake API.
 8. The administrator calls the customer to confirm their safety and coordinate roadside recovery assistance.
- **Alternate Flow:** Step 6 (No Network Coverage): The customer's mobile device loses cellular connection. The application saves the text descriptions and captured photos locally, then queues them to upload automatically once a stable connection is re-established.
 - **Postconditions:** Emergency networks are notified, the vehicle rental session is closed, the asset status changes to ACCIDENT_UNDER_INVESTIGATION, and an insurance claim file is created.

8. UML SYSTEM ARTIFACTS

8.1 Use Case Diagram (Textual Representation)

- Customer Actor interacts with: UC-1.0 (Book Vehicle), UC-4.0 (Report Accident / SOS), and Search & Filter Catalog.
- Vehicle Owner Actor interacts with: UC-2.0 (List Vehicle) and View Escrow Balance & Payouts.
- Parking Worker Actor interacts with: Inspect & Clean Asset and Complete Hub Handover Setup.
- Administrator Actor interacts with: UC-3.0 (Approve Listing), Moderate System Reviews, and Monitor Live Emergency SOS Board.
- UC-1.0 (Book Vehicle) includes Process Payment.
- UC-2.0 (List Vehicle) includes Pay Security Deposit.

8.2 Activity Diagram: Customer Booking to Return Workflow

- Start state leads to Search & Apply Catalog Filters.
- Select Specific Vehicle Asset leads to Check User Age & License Rules.
- If conditions evaluate as Invalid, trigger Show Eligibility Rejection Alert and advance to End state.

- If conditions evaluate as Valid, advance to Create Temporary 120-Sec Database Asset Lock.
- Route Checkout Session to Payment Gateway leads to Check Payment Status Result.
- If the payment state evaluates as Failed, perform Release Asset Lock and return the user to Search state.
- If payment state evaluates as Success, perform Update State to Booked and Issue Trip QR Security Token.
- Customer Unlocks & Drives Vehicle leads to Return Vehicle to Designated Hub.
- Enforce 30-Min Staging Maintenance moves system state to complete at End state.

8.3 Sequence Diagram: Real-Time Booking Processing Execution

- Customer App submits a Click Book action to the API Gateway.
- API Gateway translates Route Request parameters to the Booking Service service layer.
- Booking Service sends Select Lock commands to the Database Cluster.
- Database Cluster returns Lock Confirmed states back to the Booking Service.
- Booking Service delivers Checkout URI redirects to the API Gateway, reaching the Customer App.
- Customer App executes Handshake Payment Transaction blocks securely with the Payment Broker.
- Payment Broker passes a Process Payment token status payload to the Database Cluster.
- Payment Broker broadcasts Success Notification updates back to the Customer App interface.
- Payment Broker sends Webhook Success Event signals to the Booking Service.
- Booking Service processes Update State parameters inside the Database Cluster.

8.4 State Diagram: Vehicle Asset Lifecycle Transitions

- Initial state moves into PENDING_ONBOARDING.
- Triggering Owner Deposit Cleared & Worker Staging Confirmed moves the asset into AVAILABLE.
- Processing a User Places Reservation Request transitions the asset to RESERVED_LOCKED.

- If a Payment Dynamic Expiry occurs, the asset returns to the AVAILABLE state.
- Implementing a Payment Token Confirmed Success event shifts the asset into BOOKED.
- Performing Customer Verification Code Authenticated at Hub triggers the ACTIVE_TRIP state.
- Executing Vehicle Returned and Checked In at Destination Hub shifts the asset to POST_RENTAL_INSPECTION.
- Completing the Mandatory 30-Minute Maintenance & Check transitions the asset state back to AVAILABLE.
- If an SOS Trigger / Collision Telemetry Event occurs during an active session, the state moves to ACCIDENT_UNDER_INVESTIGATION, leading to the terminal block.

9. DATABASE REQUIREMENTS

9.1 Schema Design Relationship Descriptions

- A User can establish zero to many Booking records; each Booking belongs strictly to one customer User.
- An Owner User can possess zero to many listed Vehicle records; each Vehicle maps exactly to one Owner.
- A Vehicle can be tied to zero to many historical Booking entries; a Booking links explicitly to one physical Vehicle.
- Each successful Booking generates exactly one distinct Payment transaction line item ledger entry.
- A Vehicle streams continuous geospatial point records captured sequentially inside the GPS_Tracklog entity.

9.2 Complete Entity Attribute Mappings

- **User Entity:** user_id (UUID, Primary Key), full_name (String), email_address (String, Unique), phone_number (String), role_type (Enum), government_id_hash (String), license_number (String), date_of_birth (Date), account_status (Enum), created_timestamp (Timestamp).
- **Vehicle Entity:** vehicle_id (UUID, Primary Key), owner_id (UUID, Foreign Key), license_plate_number (String, Unique), category (Enum), hourly_rental_rate (Numeric), fuel_type (Enum), tank_capacity (Numeric), mileage_metric (Numeric), comfort_rating_cache (Numeric), manufacture_year (Integer), availability_state (Enum), current_hub_id (UUID, Foreign Key), gps_hardware_token (String, Unique).

- **Booking Entity:** booking_id (UUID, Primary Key), customer_id (UUID, Foreign Key), vehicle_id (UUID, Foreign Key), scheduled_start_time (Timestamp), scheduled_end_time (Timestamp), actual_pickup_time (Timestamp), actual_return_time (Timestamp), initial_cost_calculated (Numeric), overage_fees_applied (Numeric), booking_status (Enum).
- **Payment Entity:** payment_id (UUID, Primary Key), booking_id (UUID, Foreign Key), transaction_amount_gross (Numeric), owner_revenue_share (Numeric), platform_revenue_share (Numeric), payment_channel (Enum), gateway_reference_token (String), payment_status (Enum), settlement_timestamp (Timestamp).
- **Review Entity:** review_id (UUID, Primary Key), author_id (UUID, Foreign Key), target_vehicle_id (UUID, Foreign Key), numeric_rating (Integer), review_text (Text), upvote_count (Integer), downvote_count (Integer), is_moderated (Boolean), audit_version (Integer).
- **GPS_Tracklog Entity:** log_id (BigInt, Primary Key), vehicle_id (UUID, Foreign Key), geographic_location_point (PostGIS Geometry Point Type), current_speed_telemetry (Numeric), heading_degrees (Integer), battery_voltage_level (Numeric), log_timestamp (Timestamp).

10. REQUIREMENTS TRACEABILITY MATRIX (RTM)

- **FR-1.1 (Multi-Role Registration Interface):** Mapped to Use Cases UC-1.0 and UC-2.0. Verified by UserRegistrationTestSuite.
- **FR-1.2 (Automated ID & Age Validation):** Mapped to Use Cases UC-1.0 and UC-3.0. Verified by IdentityValidationServiceTests.
- **FR-1.3 (Motor Vehicle Eligibility Rules):** Mapped to Use Case UC-1.0. Verified by EligibilityEngineTestSuite.
- **FR-1.4 (Bicycle Eligibility Rules):** Mapped to Use Case UC-1.0. Verified by EligibilityEngineTestSuite.
- **FR-2.1 (Vehicle Metadata Display):** Mapped to Use Cases UC-1.0 and UC-2.0. Verified by CatalogRenderControllerTests.
- **FR-2.2 (Dynamic Catalog Inventory Filters):** Mapped to Use Case UC-1.0. Verified by SearchQueryCompilationTests.
- **FR-2.3 (Owner Onboarding Deposit Bonds):** Mapped to Use Case UC-2.0. Verified by OwnerFinancialEscrowTests.
- **FR-2.4 (Staging Hub Asset Onboarding):** Mapped to Use Cases UC-2.0 and UC-3.0. Verified by HubInventoryLifecycleTests.

- **FR-3.4 (Atomic Multi-Request Locking):** Mapped to Use Case UC-1.0. Verified by ConcurrencyRaceConditionTests.
- **FR-3.5 (Mandatory Post-Trip Maintenance):** Mapped to Use Case UC-1.0. Verified by AssetTurnaroundInterlockTests.
- **FR-4.3 (Mid-Transaction Payment Fault Recovery):** Mapped to Use Case UC-1.0. Verified by PaymentGatewayFallbackTests.
- **FR-4.5 (Automated Owner 80/20 Revenue Split):** Mapped to Use Cases UC-1.0 and UC-2.0. Verified by LedgerDistributionEngineTests.
- **FR-5.4 (Pro-Rata Fuel Sensor Overages):** Mapped to Use Case UC-1.0. Verified by TelemetryBillingIntegrationTests.
- **FR-6.3 (Immediate One-Touch SOS Dispatch):** Mapped to Use Case UC-4.0. Verified by EmergencyBroadcasterAlertTests.
- **FR-6.5 (Immutable Append-Only Review Logging):** Mapped to Use Case UC-3.0. Verified by ReviewAuditLedgerTestSuite.

11. ACCEPTANCE CRITERIA

AC-1.0: User Verification Performance

Given a new customer uploading identity files during registration, the system must complete automated extraction and verify age against the third-party registry within 4,000 milliseconds. It must reject registration profiles under 18 years of age attempting to view motorized vehicles.

AC-2.0: Double-Booking Prevention Under Load

Given 2,500 simultaneous checkout requests submitted for the same single vehicle identification number, the system must permit exactly one transaction to complete successfully. It must invalidate all remaining overlapping transaction attempts and return a clear failure notification to the remaining users within 1,000 milliseconds.

AC-3.0: SOS Disaster Notification Latency

Given an active vehicle driver triggering the in-app emergency SOS button, the platform must transmit the emergency packet to the administrative incident dashboard and trigger outbound alerts to emergency networks within 1,500 milliseconds of the button press.

AC-4.0: Immutable Review Guardrails

Given an administrative user or vehicle owner attempting a SQL update or API delete command on a low rating entry, the database engine must block the delete operation via database trigger functions, return an access error code, and write an incident record to the security audit ledger.

12. FUTURE ENHANCEMENTS

12.1 AI Demand Prediction and Dynamic Pricing

Future updates will introduce a machine-learning forecasting engine. This module will analyze local weather patterns, public transportation schedules, regional holidays, and historical reservation data to predict vehicle demand across individual parking hubs. The system will adjust hourly rental rates in real time, maximizing fleet utilization and optimizing platform revenue.

12.2 Predictive Maintenance and IoT Diagnostics

The system can expand its use of onboard telemetry by monitoring real-time error codes from the vehicle's diagnostic systems (OBD-II). By analyzing trends in battery degradation, brake pad wear indicators, fluid levels, and engine temperature metrics, predictive models will automatically schedule maintenance tasks before component failures occur. This approach reduces overall maintenance costs and minimizes operational downtime for fleet owners.

12.3 EV Charging Network Integration

As electric vehicles make up a larger share of the rental fleet, the system will integrate directly with public charging network APIs. This integration will enable the platform to check charging plug availability in real time, route customers to optimal charging stations during long-distance trips, and automate charging session payments using the customer's active rental profile. This removes the need for customers to manage multiple separate charging accounts.